

Report progetto 1

Simone Leonardi 247748

April 2026

Contents

1	Introduzione all' algoritmo e al problema	2
2	Descrizione del codice sequenziale	3
3	Codice in parallelo	4
4	Descrizione benchmark	7
5	Analisi risultati	8
5.1	Discussione risultati	8
5.2	Dati sperimentali	8
5.3	Analisi risultati	15
5.3.1	Effetto dell'Hyperthreading	17
5.3.2	Effetto dell'Overscheduling	17
5.4	Gestione cache	18
5.5	Scelte progettuali	19
5.5.1	Tiling	19
5.5.2	Direttive sincronizzate	19
6	Conclusioni	19

1 Introduzione all' algoritmo e al problema

In questo codice noi prendiamo in analisi un codice di pixel binning e cerchiamo di ottimizzare il suo tempo di esecuzione utilizzando tecniche di elaborazione parallela. Il pixel binning è un processo molto comune nel mondo della elaborazione delle immagini, è un processo che prevede la combinazione dei pixel adiacenti in un'immagine elaborandoli per ottenere una varietà di vantaggi. Nel nostro caso, prendiamo un cluster di pixel adiacenti (dimensione definita dalla variabile `bin_factor`); dopo di che, facciamo la media di questi pixel canale per canale (quindi per i 3 canali RGB, dato che il codice lavora su file jpg). Come risultato questo ci restituisce una immagine di risoluzione minore data dalla formula

$$MPx_{out} = \frac{MPx_{in}}{\text{bin_factor}^2}$$

dove MPx è megapixel.

Nel nostro caso analizziamo l'esempio del binning 4x4 un cluster di 16 pixel diventa un singolo pixel, questo riduce a 1/16 il numero di pixel e riducendo la risoluzione della immagine in input. Questo algoritmo ha come effetto collaterale che il pixel risultante ha un rapporto segnale rumore, molto simile al avere un "pixel pitch" (dimensione del pixel sul chip su cui avviene la acquisizione) maggiore a quella originale.

Gli algoritmi di binning sono spesso utilizzati per migliorare immagini con qualità insufficiente oppure con dimensioni troppo elevate. Una variante di questi algoritmi (spesso gestita direttamente in hardware) la troviamo nelle fotocamere mirrorless ad alta risoluzione: ad esempio, la Leica SL3 lo utilizza per ottenere un output in 4K partendo da un sensore con risoluzione nativa di 61 MP (pari a circa 9.8K).

Allo stesso modo, la tecnologia del binning è presente nella maggior parte degli smartphone che, essendo muniti di sensori Quad Bayer con risoluzioni molto alte, sfruttano questo processo per generare file più piccoli; ciò risulta particolarmente utile per ottimizzare la fase di archiviazione e gestire meglio il peso dei dati.

Questi algoritmi si prestano molto bene all'elaborazione parallela, in quanto ogni cluster di pixel di input e ogni corrispondente pixel di output sono indipendenti l'uno dall'altro. Tuttavia, l'esecuzione sequenziale su un singolo thread risulta estremamente intensiva: operando su milioni di pixel moltiplicati per i tre canali colore (RGB), ci si trova a gestire un numero elevato di operazioni semplici. Questa caratteristica rende il carico di lavoro facilmente suddivisibile tra i vari core della CPU, permettendo di distribuire lo sforzo computazionale in modo efficiente.

2 Descrizione del codice sequenziale

Il codice fondamentale si prepone l'obiettivo di implementare l'algoritmo spiegato nella sezione precedente. Ho usato la libreria `stb_image` che contiene la funzione `stb_load` che permette di caricare una immagine sottoforma di matrice di unsigned char e viene gestita come un array contiguo di byte, in cui ogni pixel occupa tre posizioni consecutive in memoria, una per ogni canale RGB (con valori che vanno da 0 a 255 perché un file jpg permette di salvare solamente 8 bit per canale per pixel).

```
unsigned char* immagine = stbi_load("input.jpg", &width,
    &height, &channels, 0);
```

In seguito creo un nuovo file vuoto per la nuova immagine e implemento il binning, inoltre dichiaro le variabili `t1` e `t2` per il calcolo del tempo di esecuzione:

```
double t1, t2;
t1= omp_get_wtime();
for (int ny = 0; ny < new_height; ny++) {
    for (int nx = 0; nx < new_width; nx++) {
        for (int c = 0; c < channels; c++) {
            int sum = 0;
            for (int dy = 0; dy < bin_factor; dy++) {
                for (int dx = 0; dx < bin_factor; dx++) {
                    int orig_x = nx * bin_factor + dx;
                    int orig_y = ny * bin_factor + dy;
                    int orig_idx = (orig_y * width +
                        orig_x) * channels + c;
                    sum += immagine[orig_idx];
                }
            }
            int out_idx = (ny * new_width + nx) *
                channels + c;
            new_image2[out_idx] = (unsigned char)(sum / (
                bin_factor * bin_factor));
        }
    }
}
t2= omp_get_wtime();
/*libero memoria*/
stbi_image_free(immagine);
delete[] new_image;
delete[] new_image2;
```

Una volta caricata l'immagine, questo frammento di codice esegue il binning dei dati contenuti nell'array lineare `immagine`. I cicli più esterni, indicizzati da `ny` e `nx`, iterano sulle coordinate di riga e colonna della nuova immagine di output, che ha dimensioni `new_height` $\times$ `new_width`. Per ogni pixel di output, il ciclo sul canale `c` calcola in modo indipendente il valore medio

di ciascun canale di colore (RGB). All'interno di questi cicli, le variabili `dx` e `dy` iterano sul blocco `bin_factor × bin_factor` che costituisce il gruppo di pixel originali corrispondente al singolo pixel di output, poiché le coordinate del pixel di input vengono recuperate come `orig_x = nx * bin_factor + dx` e `orig_y = ny * bin_factor + dy`, traslando la posizione del blocco in base all'indice del pixel di output. L'indice lineare `orig_idx` viene quindi calcolato dalle coordinate bidimensionali e dal canale come `(orig_y * width + orig_x) * channels + c`, consentendo di accedere correttamente al valore del pixel nell'array unidimensionale `immagine`. La variabile `sum`, dichiarata come `int` per evitare overflow durante l'accumulo, somma tutti i contributi dei pixel appartenenti al blocco; una volta completata la scansione, l'indice del pixel di output viene calcolato tramite `out_idx = (ny * new_width + nx) * channels + c`, e a `new_image2[out_idx]` viene assegnata la media aritmetica dei valori accumulati, ovvero `sum / (bin_factor * bin_factor)`, convertita in `unsigned char` per tornare al formato a 8 bit per canale. Pertanto, ogni nuovo pixel dell'immagine rappresenta la media dei corrispondenti pixel del blocco originale. In seguito il codice richiama le funzioni `delete[]` per liberare lo spazio allocato alla immagine (buona prassi come descritto nella libreria usata).

3 Codice in parallelo

La parallelizzazione è stata implementata utilizzando OpenMP, applicando la direttiva `#pragma omp parallel for` alla parte di codice interessata:

```
#define STB_IMAGE_IMPLEMENTATION
#define STB_IMAGE_WRITE_IMPLEMENTATION
#include "stb_image.h"
#include "stb_image_write.h"
#include <iostream>
#include <cstdlib>
#include <cstring>
#include <omp.h>

void binning_no_collapse(const unsigned char* immagine,
                        unsigned char* output,
                        int width, int height, int
                        channels, int bin_factor) {
    int new_width = width / bin_factor;
    int new_height = height / bin_factor;

#pragma omp parallel for schedule(runtime)
    for (int ny = 0; ny < new_height; ny++) {
        for (int nx = 0; nx < new_width; nx++) {
            for (int c = 0; c < channels; c++) {
                int sum = 0;
```

```

        for (int dy = 0; dy < bin_factor; dy++) {
            for (int dx = 0; dx < bin_factor; dx
++)) {
                int orig_x = nx * bin_factor + dx
;
                int orig_y = ny * bin_factor + dy
;
                int orig_idx = (orig_y * width +
orig_x) * channels + c;
                sum += imagine[orig_idx];
            }
            int out_idx = (ny * new_width + nx) *
channels + c;
            output[out_idx] = (unsigned char)(sum / (
bin_factor * bin_factor));
        }
    }
}

void binning_with_collapse(const unsigned char* imagine,
    unsigned char* output,
        int width, int height, int
channels, int bin_factor) {
    int new_width = width / bin_factor;
    int new_height = height / bin_factor;

#pragma omp parallel for schedule(runtime) collapse(2)
    for (int ny = 0; ny < new_height; ny++) {
        for (int nx = 0; nx < new_width; nx++) {
            for (int c = 0; c < channels; c++) {
                int sum = 0;
                for (int dy = 0; dy < bin_factor; dy++) {
                    for (int dx = 0; dx < bin_factor; dx
++)) {
                        int orig_x = nx * bin_factor + dx
;
                        int orig_y = ny * bin_factor + dy
;
                        int orig_idx = (orig_y * width +
orig_x) * channels + c;
                        sum += imagine[orig_idx];
                    }
                }
                int out_idx = (ny * new_width + nx) *
channels + c;
                output[out_idx] = (unsigned char)(sum / (
bin_factor * bin_factor));
            }
        }
    }
}

```

```

    }
}

int main(int argc, char* argv[]) {
int width, height, channels;
/*caricamento immagine*/
unsigned char* immagine = stbi_load("input.jpg", &width,
    &height, &channels, 0);
if (immagine == nullptr) {
    std::cerr << "Errore: impossibile caricare l'immagine\n";
    return 1;}
else {
    std::cout << "Immagine caricata: " << width << "x" <<
        height << ", canali: " << channels << std::endl;
}

/*binning*/

int bin_factor = 4; // esempio di fattore di binning
int new_width = width / bin_factor;
int new_height = height / bin_factor;
unsigned char* output = new unsigned char[new_width *
    new_height * channels];
/*fine allocazione e inizializzazione*/

const char* mode = (argc > 1) ? argv[1] : std::getenv("
    BINNING_MODE");
bool use_collapse = (mode != nullptr && std::strcmp(mode,
    "collapse") == 0);

std::cout << "Modalita binning: " << (use_collapse ? "
    collapse(2)" : "no collapse") << std::endl;

double t1, t2;
if (use_collapse) {
    t1 = omp_get_wtime();
    binning_with_collapse(immagine, output, width, height
        , channels, bin_factor);
} else {
    t1 = omp_get_wtime();
    binning_no_collapse(immagine, output, width, height,
        channels, bin_factor);
}
t2 = omp_get_wtime();
/*binning finito*/
/*stampo nuova immagine*/

stbi_write_jpg("output.jpg", new_width, new_height,

```

```

        channels, output, 100);

/*libero memoria*/
stbi_image_free(image);
delete[] output;

std::cout<< "Tempo di esecuzione del binning: " << (t2 -
        t1) << " secondi"<<std::endl<<std::endl;

return 0;
}

}

```

Il codice fa 2 prove ogni volta:una utilizzando collapse e una senza. Il numero di thread e lo scheduler, sono invece gestiti dal makefile.

4 Descrizione benchmark

I risultati del benchmark qui discussi sono ottenuti eseguendo il test su un processore Intel Core i7-11800H con 8 core e 16 thread .

I test sono condotti su immagini di varie dimensioni(20Mpx,96Mpx e 386 mpx) e con bin_factor costante per isolare l'effetto delle variabili in analisi (come carico, e fattori di parallelizzazione), variare il bin_factor avrebbe introdotto una variabile che alterava sia il carico computazionale che il grado di parallelismo disponibile.

Variabili testate:

- Numero di Thread: eseguiamo il test prima in sequenziale per stabilire una baseline e in seguito lo proviamo parallelizzato con 2,4,8,**16**,32,64,128 Thread
- Dimensioni immagine: 20Mpx,96Mpx e 386 mpx che con bin_factor=4 daranno in output immagini di 5MPx, 24MPx e 96 MPx
- Scheduler: faremo il test usando static, dynamic e guided
- Collapse: quando testiamo utilizziamo collapse(2) poiché, come dimostrato da diverse prove, valori superiori (da collapse(3) in poi) non hanno mostrato benefici e ha anzi peggiorano le prestazioni, probabilmente per aumento dell'overhead e minore località dei dati.

Per ottenere le metriche di performance utilizziamo `omp_get_time` per misurare solamente il binning escludendo tempi di I/O e come metriche di performance utilizziamo lo Speedup assoluto e l'Efficienza parallela. Queste metriche, calcolate tramite lo script Python `elaborazione.py`, permettono di valutare quantitativamente il miglioramento prestazionale:

- **Speedup Assoluto (S):** Definito come il rapporto tra il tempo di esecuzione sequenziale (T_{seq}) e il tempo di esecuzione parallela (T_{par}):

$$S = \frac{T_{seq}}{T_{par}} \quad (1)$$

- **Efficienza Parallela (E):** Indica quanto bene vengono utilizzate le risorse computazionali rispetto al numero di thread (N) impiegati:

$$E = \frac{S}{N} = \frac{T_{seq}}{N \cdot T_{par}} \quad (2)$$

Lo script `elaborazione.py` automatizza il calcolo di questi valori processando i tempi medi ottenuti durante le varie run del benchmark.

Sono state fatte 3 ripetizioni di tutti i benchmark e di seguito sono riportati i risultati.

È importante notare che la baseline sequenziale è stata ottenuta eseguendo il codice senza OpenMP, piuttosto che con `num_threads=1`. Questa scelta è motivata dal fatto che anche con un solo thread, OpenMP introduce un overhead non trascurabile dovuto all'inizializzazione del runtime e alla gestione delle regioni parallele.

I test, il calcolo delle metriche e la creazione dei grafici sono stati automatizzati tramite un codice in python e uno script che eseguiva in automatico il benchmark descritto nel `makefile`.

5 Analisi risultati

5.1 Discussione risultati

In questa sezione vengono presentati e discussi i risultati dei benchmark condotti sui tre carichi di lavoro. L'analisi è organizzata per evidenziare le configurazioni ottimali, i trend di scalabilità, l'impatto degli scheduler e della clausola `collapse`, e l'effetto della dimensione del carico.

5.2 Dati sperimentali

Table 1: Riepilogo Benchmark per carico di lavoro 20 MPx (media su 3 run).

Thread	Scheduler	Collapse(2)	Avg. Time [s]	Speedup	Efficiency	Fastest
1	sequential	–	0.029000	1.000	1.000	–
2	static	no	0.037000	0.784	0.392	
2	static	yes	0.043000	0.674	0.337	
2	dynamic	no	0.035000	0.829	0.414	
2	dynamic	yes	0.126667	0.229	0.114	
2	guided	no	0.032667	0.888	0.444	
2	guided	yes	0.040667	0.713	0.357	
4	static	no	0.024000	1.208	0.302	
4	static	yes	0.022333	1.299	0.325	
4	dynamic	no	0.018333	1.582	0.395	
4	dynamic	yes	0.106000	0.274	0.068	
4	guided	no	0.020333	1.426	0.357	
4	guided	yes	0.022333	1.299	0.325	
8	static	no	0.014667	1.977	0.247	
8	static	yes	0.015667	1.851	0.231	
8	dynamic	no	0.012000	2.417	0.302	
8	dynamic	yes	0.068667	0.422	0.053	
8	guided	no	0.012000	2.417	0.302	
8	guided	yes	0.013333	2.175	0.272	
16	static	no	0.012333	2.351	0.147	
16	static	yes	0.013000	2.231	0.139	
16	dynamic	no	0.009333	3.107	0.194	✓
16	dynamic	yes	0.046667	0.621	0.039	
16	guided	no	0.010667	2.719	0.170	
16	guided	yes	0.011333	2.559	0.160	
32	static	no	0.012667	2.289	0.072	
32	static	yes	0.012667	2.289	0.072	
32	dynamic	no	0.011666	2.486	0.078	
32	dynamic	yes	0.048667	0.596	0.019	
32	guided	no	0.012333	2.351	0.073	
32	guided	yes	0.012333	2.351	0.073	
64	static	no	0.016333	1.776	0.028	
64	static	yes	0.017333	1.673	0.026	
64	dynamic	no	0.015000	1.933	0.030	
64	dynamic	yes	0.051000	0.569	0.009	
64	guided	no	0.015333	1.891	0.030	
64	guided	yes	0.016000	1.812	0.028	
128	static	no	0.023667	1.225	0.010	
128	static	yes	0.024333	1.192	0.009	
128	dynamic	no	0.022667	1.279	0.010	
128	dynamic	yes	0.059000	0.492	0.004	
128	guided	no	0.022667	1.279	0.010	
128	guided	yes	0.023333	1.243	0.010	

Table 2: Riepilogo Benchmark per carico di lavoro 96 MPx (media su 3 run).

Thread	Scheduler	Collapse(2)	Avg. Time [s]	Speedup	Efficiency	Fastest
1	sequential	–	0.140333	1.000	1.000	–
2	static	no	0.183333	0.765	0.383	
2	static	yes	0.187667	0.748	0.374	
2	dynamic	no	0.166667	0.842	0.421	
2	dynamic	yes	0.611333	0.230	0.115	
2	guided	no	0.164667	0.852	0.426	
2	guided	yes	0.186000	0.754	0.377	
4	static	no	0.092667	1.514	0.379	
4	static	yes	0.100000	1.403	0.351	
4	dynamic	no	0.086333	1.625	0.406	
4	dynamic	yes	0.505333	0.278	0.069	
4	guided	no	0.082667	1.698	0.424	
4	guided	yes	0.101333	1.385	0.346	
8	static	no	0.058333	2.406	0.301	
8	static	yes	0.064333	2.181	0.273	
8	dynamic	no	0.052000	2.699	0.337	
8	dynamic	yes	0.351333	0.399	0.050	
8	guided	no	0.053333	2.631	0.329	
8	guided	yes	0.056667	2.476	0.310	
16	static	no	0.050000	2.807	0.175	
16	static	yes	0.055333	2.536	0.159	
16	dynamic	no	0.035667	3.935	0.246	
16	dynamic	yes	0.235000	0.597	0.037	
16	guided	no	0.038667	3.629	0.227	
16	guided	yes	0.035333	3.972	0.248	✓
32	static	no	0.054333	2.583	0.081	
32	static	yes	0.043667	3.214	0.100	
32	dynamic	no	0.037000	3.793	0.119	
32	dynamic	yes	0.224333	0.626	0.020	
32	guided	no	0.039333	3.568	0.111	
32	guided	yes	0.039333	3.568	0.111	
64	static	no	0.062667	2.239	0.035	
64	static	yes	0.044667	3.142	0.049	
64	dynamic	no	0.041333	3.395	0.053	
64	dynamic	yes	0.219667	0.639	0.010	
64	guided	no	0.043000	3.264	0.051	
64	guided	yes	0.042667	3.289	0.051	
128	static	no	0.049000	2.864	0.022	
128	static	yes	0.050000	2.807	0.022	
128	dynamic	no	0.048333	2.903	0.023	
128	dynamic	yes	0.227000	0.618	0.005	
128	guided	no	0.049000	2.864	0.022	
128	guided	yes	0.048333	2.903	0.023	

Baseline for speedup/efficiency: average sequential time on the same workload. Fastest marks indicate the minimum average execution time.

Table 3: Riepilogo Benchmark per carico di lavoro 386 MPx (media su 3 run).

Thread	Scheduler	Collapse(2)	Avg. Time [s]	Speedup	Efficiency	Fastest
1	sequential	–	0.520667	1.000	1.000	–
2	static	no	0.708667	0.735	0.367	
2	static	yes	0.693000	0.751	0.376	
2	dynamic	no	0.598000	0.871	0.435	
2	dynamic	yes	2.246333	0.232	0.116	
2	guided	no	0.630667	0.826	0.413	
2	guided	yes	0.714333	0.729	0.364	
4	static	no	0.511000	1.019	0.255	
4	static	yes	0.365000	1.426	0.357	
4	dynamic	no	0.325333	1.600	0.400	
4	dynamic	yes	1.966667	0.265	0.066	
4	guided	no	0.320333	1.625	0.406	
4	guided	yes	0.370667	1.405	0.351	
8	static	no	0.201333	2.586	0.323	
8	static	yes	0.212667	2.448	0.306	
8	dynamic	no	0.178667	2.914	0.364	
8	dynamic	yes	1.288000	0.404	0.051	
8	guided	no	0.174000	2.992	0.374	
8	guided	yes	0.206667	2.519	0.315	
16	static	no	0.144000	3.616	0.226	
16	static	yes	0.137333	3.791	0.237	
16	dynamic	no	0.127000	4.100	0.256	
16	dynamic	yes	0.848667	0.614	0.038	
16	guided	no	0.125333	4.154	0.260	
16	guided	yes	0.124000	4.199	0.262	
32	static	no	0.132000	3.944	0.123	
32	static	yes	0.139333	3.737	0.117	
32	dynamic	no	0.119333	4.363	0.136	✓
32	dynamic	yes	0.830667	0.627	0.020	
32	guided	no	0.125667	4.143	0.129	
32	guided	yes	0.125333	4.154	0.130	
64	static	no	0.140667	3.701	0.058	
64	static	yes	0.144000	3.616	0.056	
64	dynamic	no	0.124000	4.199	0.066	
64	dynamic	yes	0.827000	0.630	0.010	
64	guided	no	0.128000	4.068	0.064	
64	guided	yes	0.129333	4.026	0.063	
128	static	no	0.144000	3.616	0.028	
128	static	yes	0.139333	3.737	0.029	
128	dynamic	no	0.132333	3.935	0.031	
128	dynamic	yes	0.849667	0.613	0.005	
128	guided	no	0.132667	3.925	0.031	
128	guided	yes	0.141000	3.693	0.029	

Baseline per speedup/efficienza: tempo sequenziale medio sullo stesso carico. I segni di spunta indicano il tempo minimo di esecuzione.

Analisi Performance - Carico 20 MPx

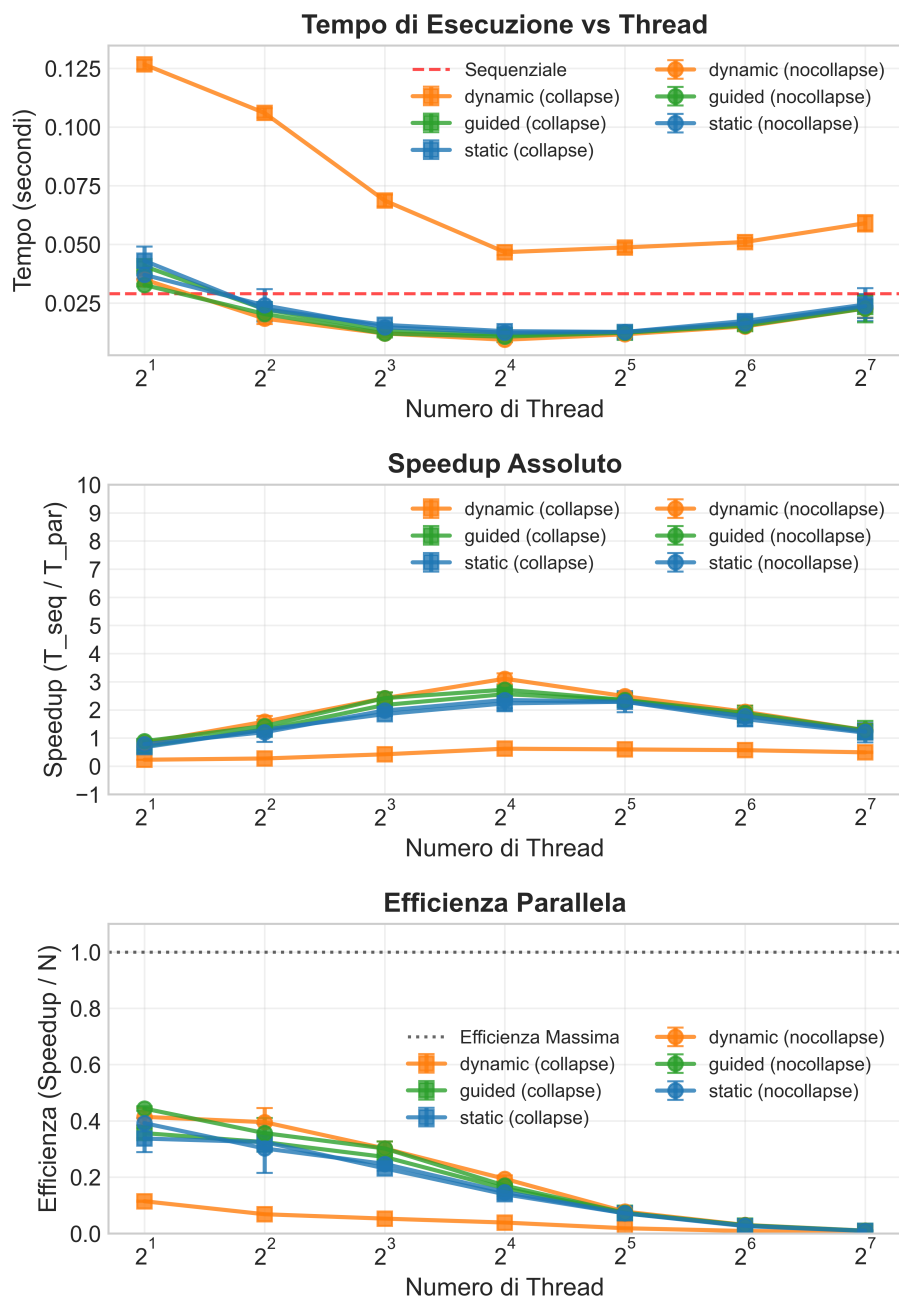


Figure 1: Analisi per 20 MPx

Analisi Performance - Carico 96 MPx

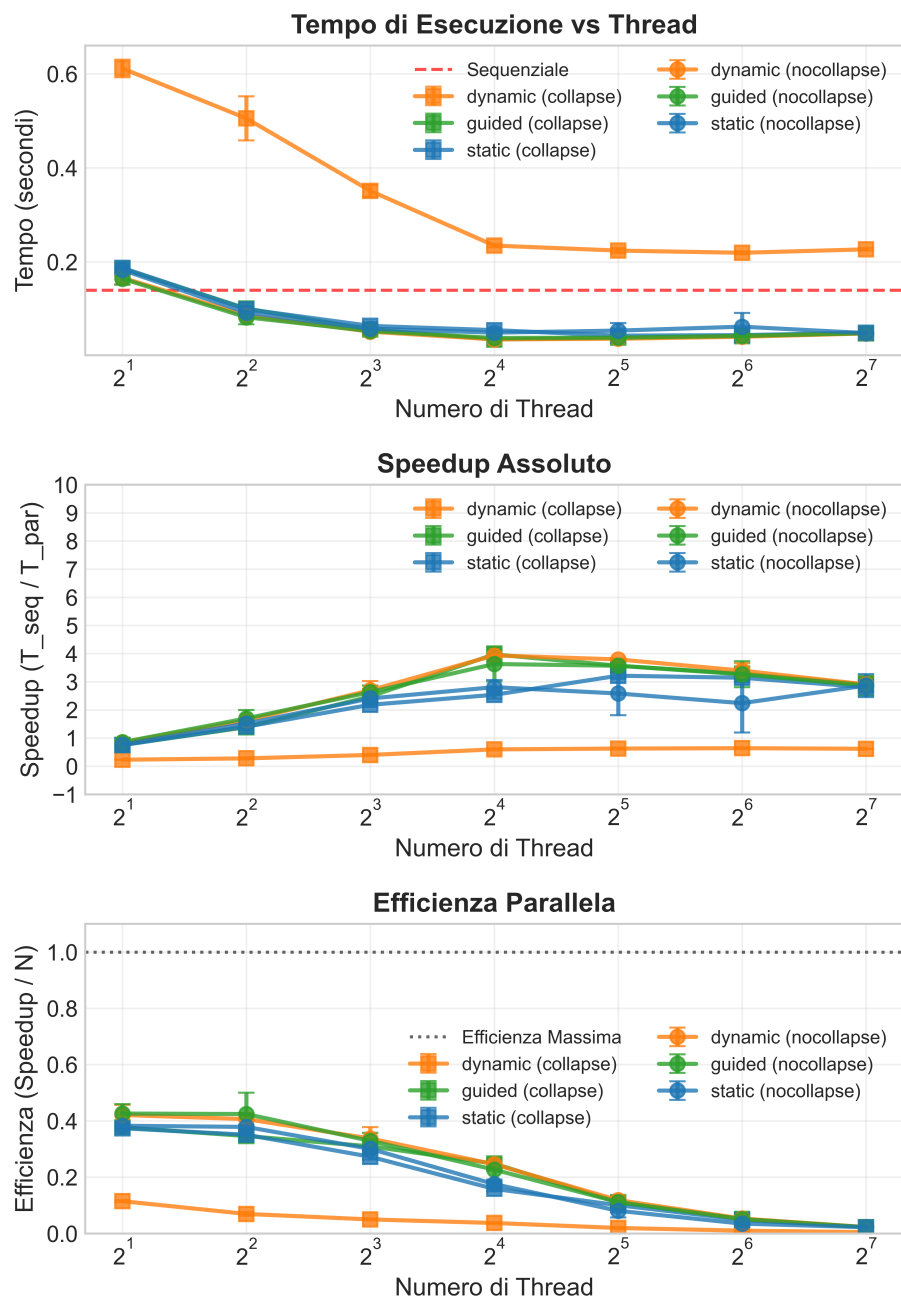


Figure 2: Analisi per 96 MPx

Analisi Performance - Carico 386 MPx

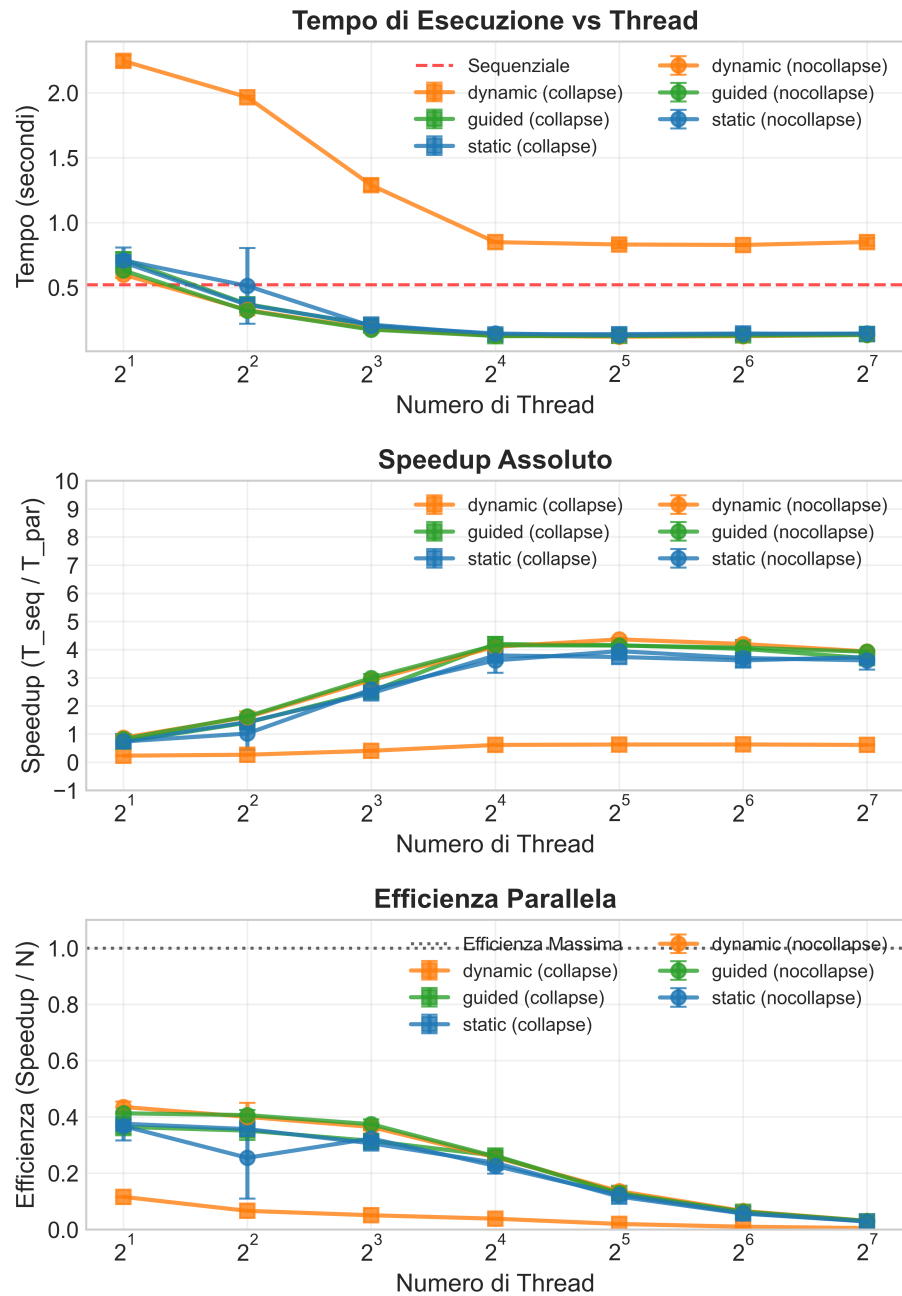


Figure 3: Analisi per 386 MPx

Variabilità dei risultati Nei grafici sono riportate le barre d’errore calcolate per le 3 ripetizioni effettuate per ogni configurazione. Possiamo osservare che nella maggior parte delle run le barre di errore sono molto piccole; sintomo di misurazioni stabili e ripetibili.

In alcuni test (come il 386 MPx con `static` a 4 thread) si notano dei tempi meno stabili. Questo è dovuto alla casualità dello scheduler nel gestire carichi così pesanti. Con blocchi di dati così grandi, anche minime variazioni negli accessi alla RAM o interferenze da parte di altri programmi possono influenzare in maniera significativa il risultato finale.

Configurazioni ottimali

La Tabella riassume le configurazioni che hanno ottenuto le migliori prestazioni per ciascun carico di lavoro

Carico	Threads	Scheduler	Collapse	Tempo [s]	Speedup	Riduzione
20 MPx	16	dynamic	NO	0.009	3.11×	68%
96 MPx	16	guided	SÌ	0.035	3.98×	75%
386 MPx	32	dynamic	NO	0.119	4.36×	77%

Table 4: Risultati ottimali per ciascun carico di lavoro

Come si può osservare, il numero ottimale di thread aumenta con la dimensione del carico: 16 threads sono sufficienti per carichi piccoli e medi, mentre per il carico più grande (386 MPx) si ottiene il massimo beneficio con 32 threads. Lo speedup massimo raggiunto varia da 3.11× per il carico più piccolo fino a 4.36× per quello più grande, con una riduzione del tempo di esecuzione compresa tra il 68% e il 77%.

5.3 Analisi risultati

Da questi risultati possiamo evincere vari trend:

1. **Speedup e Carico:** Lo speedup cresce proporzionalmente al carico di lavoro. Passando da 20 MPx a 386 MPx, lo speedup sale da 3.1x a 4.36x; questo accade perché, con un carico maggiore, l’impatto dell’overhead di parallelizzazione diventa sempre più trascurabile rispetto al tempo di esecuzione della elaborazione.
2. **Ottimizzazione per carichi medi:** Per immagini di media dimensione, la combinazione dello scheduler `guided` con l’opzione `collapse` risulta molto efficace, garantendo un ottimo bilanciamento del carico tra i thread.
3. **Efficienza sui grandi carichi:** Per i carichi più elevati (386 MPx), lo scheduler `dynamic` senza `collapse` si rivela la scelta migliore. In questi scenari, il costo di gestione del `dynamic` è ampiamente compensato dalla mole di lavoro, offrendo il miglior compromesso tra gestione e prestazioni.

4. **Oversubscription:** Nonostante la CPU disponga di 8 core, i risultati migliori a 386 MPx si ottengono con 32 thread non 16 come sarebbe con un classico hyperthreading. Essendo l'algoritmo fortemente *memory-bound*, i core subiscono spesso stalli in attesa dei dati dalla DDR. Utilizzando 32 thread, si minimizzano i tempi morti: mentre un thread attende i dati, un altro può occupare il core per eseguire i calcoli.
5. **Limiti della memoria:** Nonostante la netta riduzione dei tempi di esecuzione grazie al parallelismo, il vero collo di bottiglia rimane la velocità di accesso alla memoria, che impedisce incrementi prestazionali ulteriori.
6. **Saturazione delle risorse:** L'efficienza cala e i tempi aumentano quando si superano i 16 o 32 thread. Ciò avviene perché l'overhead e la contesa sulle risorse superano i benefici derivanti dalla parallelizzazione.
7. **Rapporto Speedup/Efficienza:** Si nota che l'efficienza parallela decresce rapidamente all'aumentare dei thread. Anche se lo speedup totale aumenta, il rendimento di ogni singolo thread cala, confermando la presenza di un overhead non trascurabile e i limiti fisici della banda di memoria.
8. **Degradazione con thread elevati:** L'uso di un numero di thread eccessivo (64, 128) porta a un netto peggioramento delle performance; questo accade perché i thread aggiuntivi, invece di velocizzare il calcolo, aumentano solo la contesa di memoria e l'overhead di OpenMP.
9. **Effetti negativi del Collapse:** L'opzione `collapse` spesso non porta benefici poiché aumenta l'overhead per la gestione degli indici e compromette la località dei dati. Questo causa un aumento dei *cache miss*, costringendo il processore a leggere più spesso dalla DDR invece che dalla cache.
10. **Configurazione consigliata:** I dati suggeriscono che per carichi piccoli e medi la scelta ideale sia utilizzare i 16 thread, mentre per carichi molto grandi conviene spingersi fino a 32 thread per mitigare i tempi di attesa della memoria.
11. **Impatto degli Scheduler:** L'influenza dello scheduler diminuisce all'aumentare del carico. Sebbene il bilanciamento rimanga importante, l'accesso alla memoria domina l'esecuzione, rendendo i vantaggi di `static`, `dynamic` e `guided` meno marcati sui grandi volumi di dati.
12. **Fenomeno dell' Overhead:** In alcuni casi con pochi thread, l'esecuzione parallela è più lenta di quella sequenziale. Questo accade perché il tempo risparmiato con la parallelizzazione non è sufficiente a coprire l'overhead richiesto per suddividere il lavoro e creare i thread.
13. **Analisi Critica: Dynamic + Collapse:** La performance peggiore in assoluto si riscontra con la combinazione `dynamic + collapse` sui 20 MPx.

L'overhead cumulativo della gestione dinamica e del ricalcolo degli indici è troppo massiccio per un carico così ridotto: quello che succede è che questo insieme di impostazioni rompe la sequenzialità degli accessi e quindi peggiora la possibilità di avere un cache hit, in quanto i thread accedono a zone di memoria non vicine tra loro. Questo costringe il sistema a leggere più spesso dalla RAM. Al contrario, lo **static** (che divide il lavoro all'inizio) o il **guided** (che gestisce chunk più grandi inizialmente) riescono a limitare meglio gli accessi frequenti alla RAM e la sincronizzazione.

L'algoritmo di pixel binning è fortemente limitato dalla memoria piuttosto che dalla capacità computazionale della CPU. Per ogni pixel di output:

- **Operazioni di memoria:** $\text{bin_factor}^2 \times 3$ letture ($16 \times 3 = 48$ byte letti per pixel output)
- **Operazioni aritmetiche:** $\text{bin_factor}^2 \times 3$ addizioni + 3 divisioni (circa 51 operazioni)

Quindi per ogni operazione c'è poco meno che un accesso in memoria. Questo spiega perché:

1. Lo speedup massimo è limitato a $4.4\times$ nonostante 16 core logici
2. L'hyperthreading e l'overscheduling forniscono benefici significativi per i carichi molto grandi (nasconde latenze memoria)
3. Oltre 32 threads le performance degradano in quanto saturiamo la memoria

5.3.1 Effetto dell'Hyperthreading

Confronto tra 8 threads (core fisici) e 16 threads (con HT):

Table 5: Beneficio dell'hyperthreading (8 vs 16 thread)

Carico	Speedup 8 thr	Speedup 16 thr	Miglioramento	Beneficio HT
20 MPx	2.42	3.11	+0.69	+28.5%
96 MPx	2.70	3.97	+1.27	+47.0%
386 MPx	2.99	4.20	+1.21	+40.5%

L'hyperthreading fornisce benefici crescenti con il carico, confermando la natura memory-bound: i thread logici aggiuntivi riempiono gli stalli di memoria.

5.3.2 Effetto dell'Overscheduling

Confronto tra 16 threads (hyperthreading) e 32 threads (overscheduling):

L'overscheduling mostra un comportamento fortemente dipendente dal carico di lavoro:

Table 6: Beneficio dell'overscheduling

Carico	Speedup 16 thr	Speedup 32 thr	Miglioramento	Beneficio Overscheduling
20 MPx	3.11	2.49	-0.62	-19.9%
96 MPx	3.97	3.79	-0.18	-4.5%
386 MPx	4.20	4.36	+0.16	+3.8%

- Per carichi piccoli (20 MPx) e medi (96 MPx), l'introduzione di thread extra genera un overhead superiore al beneficio computazionale, portando a un peggioramento delle performance (fino al -20%).
- Per il carico massimo (386 MPx), si osserva un'inversione di tendenza con un beneficio del 3.8%.

Questo fenomeno conferma ulteriormente la natura *memory-bound* del problema. Nel caso dei 386 MPx, la mole di dati è tale da generare stalli di memoria così frequenti che perfino l'Hypertreading non riesce a saturare completamente i core fisici. In questo scenario specifico, l'uso di 32 thread permette al sistema operativo di "nascondere" le latenze della RAM più efficacemente: mentre un gruppo di thread è bloccato in attesa della DDR, l'altro può subentrare nell'esecuzione.

Tuttavia, il beneficio dell'overscheduling è marginale rispetto a quello dell'HT e scompare rapidamente non appena il rapporto tra tempo di calcolo e tempo di accesso alla memoria diventa più bilanciato (carichi minori).

5.4 Gestione cache

Come discusso sopra l'algoritmo di binning è fortemente influenzato dalla memoria. Il processore usato possiede 8 core con 16 thread logici, di una cache L1 da 640 KB (80 KB per core), una da L2 da 10 MB (1.25 MB per core) e una L3 da 24 MB (3 MB per core).

Le immagini prese in analisi arrivano a 386 MPx; considerando 3 byte per pixel (RGB a 8 bit), la quantità di dati da processare è anche di svariate centinaia di MB; quindi molto maggiore della dimensione della cache; nel caso specifico della nostra analisi la immagine da 20 MPx è di 16,8MB, quella da 96 MPx è da 63,4MB e quella da 386 MPx è da 116,7MB. Le dimensioni riportate (16.8 MB, 63.4 MB, 116.7 MB) si riferiscono alla dimensione dei file su disco in formato compresso (JPG). Tuttavia, durante l'esecuzione, le immagini vengono completamente decomprese in memoria.

Considerando che ogni pixel è rappresentato da 3 byte (RGB), le dimensioni effettive degli array in RAM sono significativamente maggiori:

- 20 MPx $\approx$ 60 MB
- 96 MPx $\approx$ 288 MB
- 386 MPx $\approx$ 1.16 GB

Questo implica che i dataset utilizzati eccedono ampiamente la capacità delle cache L1, L2 e L3, rendendo il problema fortemente memory-bound e giustificando i comportamenti osservati nelle prestazioni. Questo significa che non è possibile mantenere l'intero dataset nella cache e che abbiamo un numero molto elevato di accessi alla RAM. Tuttavia i pixel vengono letti sequenzialmente e su blocchi adiacenti; quindi si mantiene una buona località spaziale: questo riduce in parte i cache miss tuttavia non elimina il problema principale. Mano a mano che aumentano i thread la cache diventa una risorsa sempre più contesa tra i thread (in quanto il loro carico è sempre più alto), più thread accedono contemporaneamente a zone diverse della cache; diminuendo le informazioni utili per ogni thread ad un dato momento nella cache e aumentando gli accessi alla RAM. Per questo motivo, oltre un certo numero di thread (circa 16–32), le performance non migliorano più e in alcuni casi peggiorano, a causa della saturazione della banda di memoria e dell'aumento dei cache miss.

5.5 Scelte progettuali

5.5.1 Tiling

In questo esercizio non è stata implementata la tecnica del tiling in quanto l'algoritmo considerato non si presta a questo tipo di ottimizzazione. Il pattern di accesso ai dati del pixel binning è infatti regolare e caratterizzato da una buona località spaziale: i pixel vengono letti in blocchi contigui in memoria, riducendo naturalmente i cache miss. L'analisi delle prestazioni ha evidenziato come il vero collo di bottiglia del sistema sia la banda di memoria piuttosto che la latenza o la capacità della cache. In scenari di questo tipo (memory-bound), il tiling non riduce il numero totale di accessi alla memoria principale, poiché ogni pixel viene comunque letto una sola volta; di conseguenza, non apporterebbe benefici significativi in termini di prestazioni.

Al contrario, l'introduzione del tiling comporterebbe un aumento della complessità del codice e un overhead aggiuntivo nella gestione dei blocchi, con un possibile impatto negativo sulle prestazioni complessive.

5.5.2 Direttive sincronizzate

Non ho utilizzato direttive di sincronizzazione come `atomic`, `critical` o `reduction`, in quanto il problema è applicato solo su dati paralleli: ogni thread opera su dati indipendenti e scrive in posizioni distinte dell'array di output. L'introduzione di direttive avrebbe solo aggiunto overhead senza benefici.

6 Conclusioni

I risultati evidenziano il potenziale della parallelizzazione per un algoritmo di pixel binning. Abbiamo ottenuto speedup significativi, fino a $4.36\times$ rispetto alla versione sequenziale, con riduzioni del tempo di esecuzione fino al 77%. Il fattore principale che influenza le performance in questo caso non è la potenza

di calcolo ma gli accessi alla memoria: il binning richiede poche operazioni aritmetiche relativamente agli accessi in memoria richiesti; mentre, al contempo le dimensioni dei file elaborati superano abbondantemente le dimensioni della cache rendendo l'accesso alla RAM il principale collo di bottiglia. Questo spiega sia la saturazione dello speedup oltre un certo numero di thread, sia il beneficio osservato con l'hyperthreading e con un moderato overscheduling, che aiutano a nascondere la latenza della memoria.

I risultati evidenziano che:

- La configurazione ottimale dipende fortemente dalla dimensione del carico
- Scelta dello scheduler in funzione del carico: i risultati sperimentali mostrano che non esiste uno scheduler universalmente ottimale, ma la scelta dipende dalla dimensione del workload:
 - Per carichi piccoli (20 MPx), **static** risulta il più rapido, poiché l'overhead di assegnazione dinamica delle iterazioni non è giustificato su lavori di breve durata.
 - Per carichi medi (96 MPx), **guided** combinato con **collapse** ottiene le migliori prestazioni, offrendo un buon compromesso tra bilanciamento del carico e overhead di scheduling.
 - Per carichi grandi (386 MPx), **dynamic** diventa preferibile, in quanto il bilanciamento dinamico ripaga il proprio overhead su iterazioni lunghe e potenzialmente sbilanciate.
- L'uso di **collapse(2)** con **dynamic** deve essere evitato
- Il workload è fortemente memory-bound, limitando lo speedup massimo raggiungibile
- L'hyperthreading fornisce benefici significativi (30-45%) per carichi significativi.

Le scelte progettuali adottate (assenza di tiling e di direttive di sincronizzazione) sono state motivate dalla natura stessa dell'algoritmo adottato: l'algoritmo presenta già una buona località spaziale e ogni thread scrive su posizioni di output distinte, quindi non sono presenti race condition e l'introduzione di ulteriori strutture avrebbe solo aggiunto overhead senza alcun beneficio significativo.

Dai risultati dei test possiamo evincere le configurazioni migliori per il sistema su cui è stato condotto il test:

Table 7: Configurazioni raccomandate per diversi scenari

Scenario	Configurazione	Speedup atteso
Carico piccolo (minore di 50 MPx)	16 threads, dynamic, no collapse	3.0–3.5×
Carico medio (tra 50 e tra 200 MPx)	16 threads, guided, no collapse	3.5–4.0×
Carico grande (maggiore di 200 MPx)	32 threads, dynamic, no collapse	4.0–4.5×
Massima efficienza	4 threads, guided, no collapse	1.5–1.7×

Le configurazioni riportate riflettono il comportamento osservato nei benchmark e sono giustificate dalla natura memory-bound dell’algoritmo. Per carichi piccoli, il dataset entra parzialmente nelle cache più veloci e l’overhead di scheduling dinamico supera il beneficio: 16 thread con **dynamic** rappresentano il giusto compromesso tra parallelismo e costo di gestione. Per carichi medi, la combinazione **guided** senza **collapse** bilancia bene il lavoro tra i thread mantenendo la località spaziale degli accessi. Per carichi grandi, la banda di memoria diventa il vero collo di bottiglia: 32 thread con **dynamic** permettono di nascondere le latenze della DDR sfruttando l’overscheduling, mentre il bilanciamento dinamico compensa le eventuali irregolarità nel tempo di completamento delle iterazioni. La configurazione a massima efficienza (4 thread) è invece consigliata quando il consumo energetico o la disponibilità di core è limitata: lo speedup è inferiore ma il rendimento per thread rimane ottimo.

Principali limitazioni:

- Il collo di bottiglia della memoria limita lo speedup massimo
- L’efficienza parallela decresce rapidamente oltre i 16 threads
- L’overhead di parallelizzazione è significativo per carichi molto piccoli